Tokenization from Scratch: From a Regex Splitter to GPT-2's BPE

Part 1 of my "Building an LLM from Scratch" series

Before a language model can learn anything, it has to turn text into numbers. That first stage is the **tokenizer**, and it is where every LLM pipeline begins. In this post I build one from scratch in Python, hit its limits, and then switch to the same Byte Pair Encoding (BPE) approach that GPT-2 and GPT-3 use, through `tiktoken`.

I'm following *Build a Large Language Model (From Scratch)* by Sebastian Raschka, and this post is my own walkthrough of the tokenization chapter.

The big picture

GPT-style models are **decoder-only** transformers. They never see raw text. The pipeline looks like this:

1. Raw text
2. Tokens (words, sub-words, punctuation)
3. Token IDs (integers)
4. Embeddings, then the model

This post covers steps 1 to 3.

Step 1: Get some text

I used a short story, *The Verdict*, as a small practice dataset.

```
import urllib.request

url = ("https://raw.githubusercontent.com/rasbt/"
       "LLMs-from-scratch/main/ch02/01_main-chapter-code/"
       "the-verdict.txt")
file_path = "the-verdict.txt"
urllib.request.urlretrieve(url, file_path)

with open(file_path, "r", encoding="utf-8") as f:
    text = f.read()

print("Total number of characters:", len(text))   # 20479
print(text[:99])
```

The file has 20,479 characters. Now we need to split it into tokens.

Step 2: Split text into tokens with regex

Start simple: split on whitespace.

```
import re

sample = "my name is vivek and i love to read history books"
tokens = re.split(r'(\s)', sample)
print(tokens)
```

The parentheses in the pattern keep the separators, so the result is full of `' '` entries. Next, split on punctuation as well, then drop the whitespace items:

```
result = re.split(r'([,.]|\s)', sample)
result = [item for item in result if item.strip()]
print(result)
```

A more complete pattern handles more punctuation and the double dash `--`:

```
text1 = "Hello, world. Is this-- a test?"
result2 = re.split(r'([,.:;?_!"()\']|--|\s)', text1)
result2 = [item.strip() for item in result2 if item.strip()]
print(result2)
# ['Hello', ',', 'world', '.', 'Is', 'this', '--', 'a', 'test', '?']
```

Tip: don't lowercase everything. Capitalisation carries information, and keeping it helps the model tell proper nouns and sentence starts from ordinary words.

Running the same pattern on the full story gives **4,690 tokens**.

```
clean_text = re.split(r'([,.:;?_!"()\']|--|\s)', text)
clean_text = [item.strip() for item in clean_text if item.strip()]
print(len(clean_text))   # 4690
```

Step 3: Build a vocabulary (token to ID)

A model needs integers. So we collect the unique tokens, sort them, and assign each an ID.

```
all_words = sorted(set(clean_text))
vocab_size = len(all_words)
print(vocab_size)   # 1130

vocab = {token: integer for integer, token in enumerate(all_words)}
```

The vocabulary has **1,130 unique tokens**. Punctuation comes first in sorted order (`'!'` is 0, `'"'` is 1, and so on), followed by words.

Step 4: A simple tokenizer class

A tokenizer needs two directions: `encode` (text to IDs) and `decode` (IDs back to text).

```
class SimpleTokenizer:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        tokens = [t.strip() for t in tokens if t.strip()]
        return [self.str_to_int[t] for t in tokens]

    def decode(self, ids):
        text = " ".join(self.int_to_str[i] for i in ids)
        # remove the space before punctuation
        return re.sub(r'\s+([,.?!"()\'])', r'\1', text)
```

Testing it:

```
tokenizer = SimpleTokenizer(vocab)
text = """It's the last he painted, you know,"
 Mrs. Gisburn said with pardonable pride."""
ids = tokenizer.encode(text)
print(ids)
print(tokenizer.decode(ids))
```

It works, with small quirks. The decoder output came back as `It' s the last he painted...`, so the apostrophe spacing isn't perfectly restored. More importantly, there is a bigger problem.

The problem: unknown words

Try encoding a word that wasn't in the story, like `"Hello"`. You get a `KeyError`. The tokenizer only knows the 1,130 tokens it has seen.

Step 5: Special context tokens

The fix is to add special tokens to the vocabulary:

- `<|unk|>` for any word not in the vocabulary
- `<|endoftext|>` to separate unrelated documents or texts

```
all_tokens = sorted(list(set(clean_text)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])
vocab = {token: integer for integer, token in enumerate(all_tokens)}
print(len(vocab))   # 1132
```

(Small bug to watch for: in my first notebook I typed `"<|unk|"` without the closing `>`. The token in the vocab must exactly match the one used in `encode`, otherwise you get another KeyError.)

Now update `encode` to fall back to `<|unk|>`:

```
class SimpleTokenizerV2:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        tokens = [t.strip() for t in tokens if t.strip()]
        tokens = [t if t in self.str_to_int else "<|unk|>" for t in tokens]
        return [self.str_to_int[t] for t in tokens]

    def decode(self, ids):
        text = " ".join(self.int_to_str[i] for i in ids)
        return re.sub(r'\s+([,.?!"()\'])', r'\1', text)
```

Test on two unrelated sentences joined with the end-of-text token:

```
text1 = "Hello, do you like tea?"
text2 = "In the sunlit terraces of the palace."
text = "<|endoftext|>".join((text1, text2))

tokenizer = SimpleTokenizerV2(vocab)
print(tokenizer.encode(text))
```

Words like "Hello", "tea" and "sunlit" are not in the story, so they all map to the `<|unk|>` ID (1131 in my run). No more crashes, but we lose information. Every unknown word looks the same to the model.

Step 6: Byte Pair Encoding (what real LLMs use)

The word-level approach has two big weaknesses: a huge vocabulary if you train on real data, and `<|unk|>` for anything new. **Byte Pair Encoding (BPE)** fixes both by working with **sub-word units**. Rare or unseen words are broken into smaller pieces (down to single characters or bytes), so nothing is ever truly "unknown".

Writing BPE from scratch is quite involved, so the practical route is `tiktoken`, OpenAI's open-source tokenizer library, which provides the GPT-2 tokenizer.

```
!pip install tiktoken

import tiktoken

tokenizer = tiktoken.get_encoding("gpt2")

text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
ids = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print(ids)
# [15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250,
8812, 2114]

print(tokenizer.decode(ids))
# Hello, do you like tea? <|endoftext|> In the sunlit terraces
```

Three things to notice:

1. `<|endoftext|>` maps to ID **50256**, the last ID in GPT-2's vocabulary of 50,257 tokens.
2. "sunlit" and "terraces" were unknown to my simple tokenizer, but BPE handles them by splitting them into known sub-word pieces.
3. Decoding gives back the exact original text, and no `<|unk|>` was needed.

Simple tokenizer vs. BPE

	Regex word tokenizer	BPE (GPT-2)
Unit	Whole words and punctuation	Sub-words
Unknown words	Need `<	unk
Vocabulary	Grows with the data	Fixed (50,257 for GPT-2)
Good for	Learning the concept	Real models

Key takeaways

- Models need numbers, so text goes through tokenization, then token IDs.
- A basic tokenizer needs a vocabulary plus `encode` and `decode`.

- Special tokens like `<|unk|>` and `<|endoftext|>` handle unknown words and document boundaries.
- Production LLMs use BPE, which avoids the unknown-word problem by using sub-words.

Next up in the series: **data sampling with a sliding window**, and then token embeddings.

Tags to use on Medium: Large Language Models, NLP, Machine Learning, Python, Tokenization